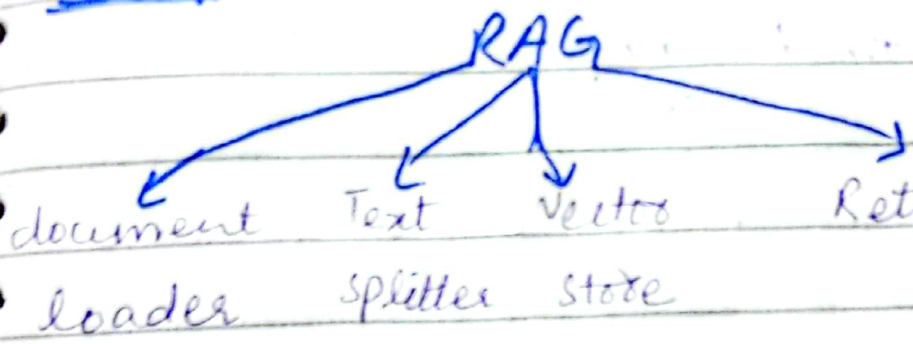
LEC-1

RAG:



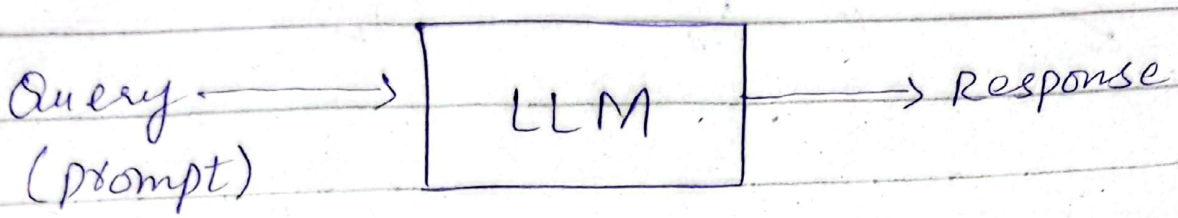
① **Document Loader:**
we can load any data from any source with the help of document loader.

② **Text splitters.:** To divide a large text into chunks

③ **Vector store:** convert text into embeddings and store it.

④ **Retrievers:** perform systematic research ~~with~~ in vector store.

Why we need Rag?



- we pre-train LLM's on huge amount of data. (e.g: Internet) & store their knowledge in parameters (in the form of numbers) known as **parametric knowledge.**
- The more powerful LLM is that have more parameters. (e.g: 17 billion, 13 billion) etc.

→ **How we access that knowledge as a user?**

- By prompting.
- The LLM search it ~~or~~ and print the answer word by word.

→ When LLM fails?

① Asking about private data.
② Asking about recent data on which the LLM is not train yet.
③ Hallucination (give incorrect info confidentially).

Fine-Tunning:

① Pick a pre-train model. (huge data)
↓
② train it again on a smaller (domain specific) data.

Two types of fine-tunning:

① **Supervised:**
labeled data set
prompt + desired output.

② **Continued pre-training:**
unsupervised-data (not labeled)

- **Supervised pre-training:**

① Collect-Data : prompt + desired output.

② Choose a method : full parameter, LORA/Qlora

③ Train for a few ephocs :
all weights or few weights py train krna hai ya ni.

④ Evaluate ky safety test :
How the model response ky its quality (we compare).

→ **Major problems in fine-tunning:**

① Trainning a huge model is costly.
② You need proper technical expertise.
③ Updating the info fastly.

To solve this we used ↙

→ **In context learning:**

It is a core capability LLM like GPT-3/4, Claude, Lama where the

model learn to solve a task purely by seeing examples in prompt — without updating its weights.

(ہم prompt میں ایسی example دیتے ہیں کہ
کسی given task کو کیسے کرنا ہے تو ان سے
LLM کی سیکھ جاتا ہے)

- Example:
- Text: This app crashes a lot → positive

→ **Emergent property:**

It is a behaviour and ability that suddenly appears in a system when it reaches a certain scale or complexity — even though it was not explicitly programmed or expected from the individual components.

(scientists نے پہلے نہیں سوچا تھا کہ model
خود In-context learning کا feature show کرے گا)

∴ So this feature automatically appear in the LLMs.

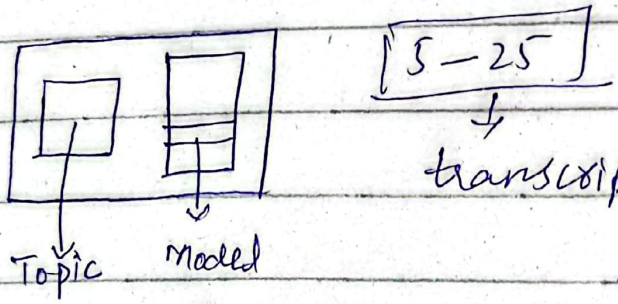
- Example:
  - GPT-1 42 → (donot provide correct guarantee output)
  - GPT-3 (175 B) ↓ parameter pr train ha.
    ↓
    (Give granted output)

paper is published name:

"Language models are few-shot learners"

→ Injecting context in prompt → RAG:

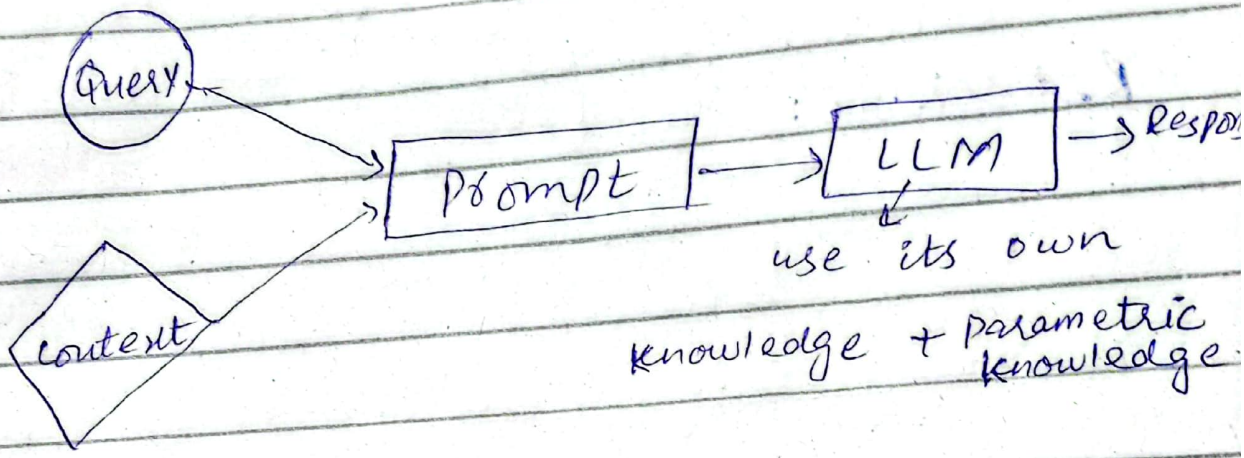
Sending 'content' in prompt instead of examples:



جب آپ اس نے issue میں Topic کو کسی Student کے Topic اس ~~model~~ اب - جو سے model کہ model کو دونوں query کے user اور transcript as a context کے ساتھ۔

What is RAG?

RAG is a way to make language model (like chatgpt) smarter by giving it extra information at the time you ask your question.



Example:

"You are a helpful assistant.
Answer the question ONLY from the provided content. If the content is insufficient, just say you don't know.

{content}

Question: {question}"

Understanding RAG:

RAG → Info retrieval + text generation

① Indexing ② Retrieval
③ Augmentation ④ Generation

Indexing: Creating external K-B ya external knowledge Base → content.

Retrieval:

- Extracting content from external knowledge Base.
- Checking only those chunks of data jo user ki query sy match karty ha.

Augmentation:

[User Query + Retrieved content] → prompt generate

Generation:

generate answers.

① Indexing: (creating knowledge Base)

① Document Ingestion: You load your source knowledge into memory.
(e.g: server, google drive)

→ Tools:
Langchain loaders (pdf loader, youtube loader etc).

② Text Chunking:
↳ Break large document into small, sematically meaningful chunks.
(E.g: 2 hr video → [1] [2] [3] chunks.)
• Tools: (RecursiveCharacterTextSplitter)

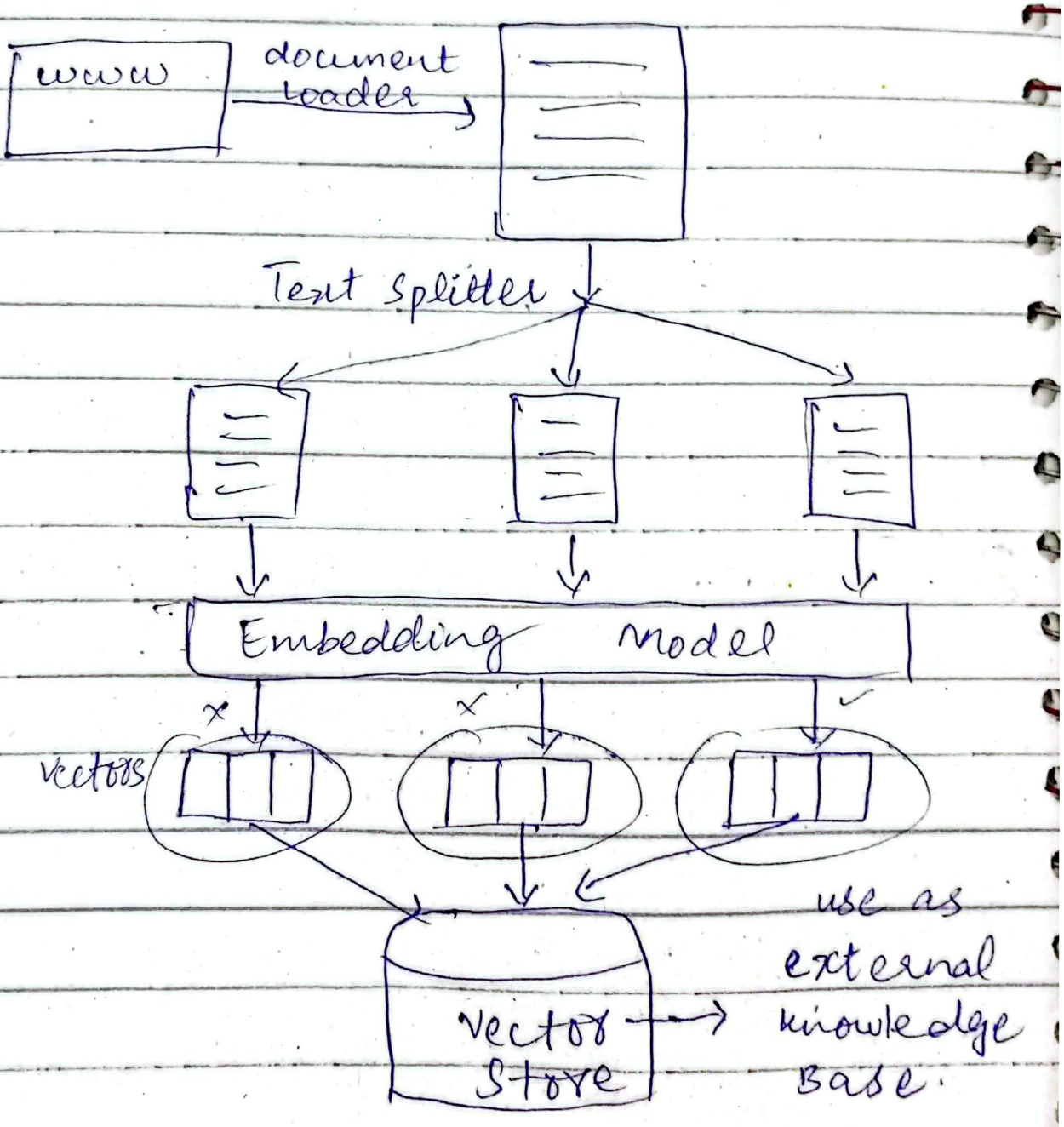
③ Embedding Generation:
↳ convert each chunk into a dense vector that captures its meaning.
→ Semantic search → always in embedding vectors
match ki part (video)
(query ki user

④ Store in vector store:

↳ store the vector along with original chunk text + metadata in a vector data base.

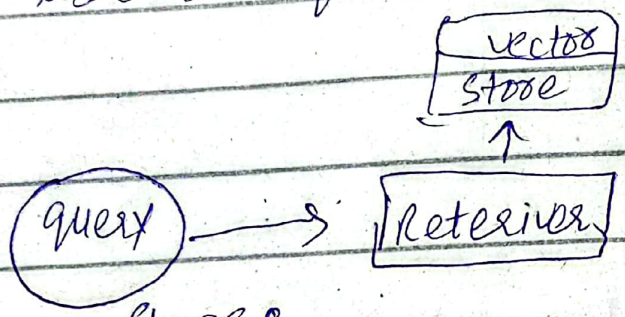
→ local : FAISS, Chroma

→ cloud : Pinecone, Qdrant.



② Retrieval :

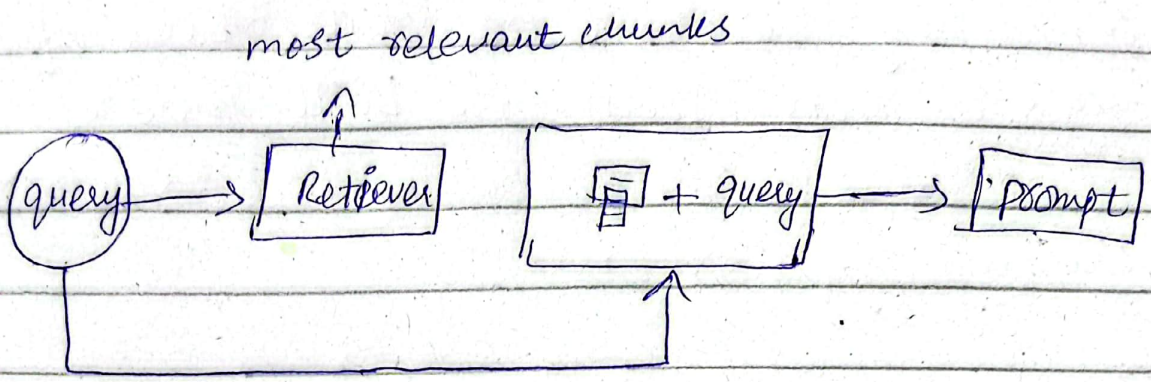
- Decide which vector is best option to become context.
- ∴ Finding the most relevant pieces of information from pre-built index (created during indexing) based on user's question.



Steps :

① convert query → embedding vector.
② Search the closet vector.
③ Perform Ranking on that vectors.
④ Fetch Top result text chunks that is called context.

③ Augmentation:



④ Generation:

- It is final step where a large LLM uses the user's query & retrieved & augmented content to generate a response.